INTERVIEW PREP SERIES

OpenCV

Interview Questions & Easy Answers

---

52 questions · 18 marked ★ most asked

Covers image basics, colour spaces, reading and writing images and video, resizing and rotating, blurring, thresholding, edge detection, morphology, contours, Hough transforms, feature detection, face detection, video processing and integration with deep learning. Code uses Python (`import cv2`, `import numpy as np`).

How to use this document

- Questions go from basic to advanced inside each section. Freshers should master the ★ ones first.
- Answers are written in simple words, the way you would say them in an interview. Learn the idea, not the exact sentences.
- Questions are based on technical rounds reported by candidates on GeeksforGeeks, LeetCode Discuss, Glassdoor, Naukri and similar interview-experience pages, plus the standard question banks interviewers draw from.
- Read one section a day and say each answer out loud once. Speaking it is what makes it stick.

Basics

Q1. What is OpenCV? ★ Most asked

OpenCV (Open Source Computer Vision Library) is a free library with 2,500+ algorithms for image processing and computer vision. It's written in C++ and has Python, Java and JavaScript bindings. It's used for face detection, object tracking, document scanning, number-plate recognition, medical imaging and robotics.

Q2. How is an image represented in OpenCV? ★ Most asked

As a **NumPy array** of pixels. A grayscale image is 2D (height × width), each value 0-255. A colour image is 3D (height × width × 3 channels). `img.shape` gives `(rows, cols, channels)` and `img.dtype` is usually `uint8`.

Q3. Why does OpenCV use BGR instead of RGB? ★ Most asked

For historical reasons: early camera makers and Windows used BGR byte order, and OpenCV kept it. So images loaded by `cv2.imread` are in BGR. Before showing them with matplotlib (which expects RGB), convert: `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.

Q4. How do you read, display and save an image? ★ Most asked

```
import cv2
img = cv2.imread("cat.jpg")              # BGR image, or None if path is wrong
gray = cv2.imread("cat.jpg", cv2.IMREAD_GRAYSCALE)
cv2.imshow("Window", img)
cv2.waitKey(0)                           # wait for a key press
cv2.destroyAllWindows()
cv2.imwrite("out.png", img)
```

Q5. What does `cv2.waitKey()` do?

It waits for a key press for the given milliseconds and returns the key code. `waitKey(0)` waits forever. Without it, the `imshow` window won't refresh. In video loops, `waitKey(1)` gives about 1 ms per frame and lets you check for a quit key like `q`.

Q6. What happens if `imread` can't find the file?

It does not throw an error; it returns `None`. Using it later causes errors like "NoneType has no attribute shape", so always check `if img is None:`.

Q7. How do you access and change pixel values?

```
b, g, r = img[100, 50]        # pixel at row 100, column 50
img[100, 50] = [0, 0, 255]     # make it red
img[0:100, 0:100] = 0          # black square (ROI)
```

Remember: index as `[row, col]` = `[y, x]`.

Q8. What is a Region of Interest (ROI)?

A cropped part of the image you want to work on, taken with NumPy slicing: `roi = img[y1:y2, x1:x2]`. Processing only the ROI is faster and more accurate.

Colour spaces

Q9. How do you convert colour spaces and why use HSV? ★ Most asked

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hsv  = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

HSV separates colour (**Hue**) from brightness (**Value**) and intensity (**Saturation**). This makes colour detection much more reliable under different lighting than BGR.

Q10. How do you detect a specific colour (e.g. blue) in an image? ★ Most asked

Convert to HSV, define a range, create a mask with `inRange`, then apply it.

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
lower = np.array([100, 150, 50])
upper = np.array([130, 255, 255])
mask = cv2.inRange(hsv, lower, upper)        # white where blue
result = cv2.bitwise_and(img, img, mask=mask)
```

Q11. What is a mask?

A binary image (0 and 255) of the same size as the image that marks which pixels to keep. Used with `bitwise_and` to extract objects or apply effects only in certain areas.

Q12. What is the difference between `cv2.add` and `cv2.bitwise_and`?

`cv2.add` adds pixel values with saturation (values above 255 are clipped to 255), which is used for brightening or blending. `bitwise_and` does a bit-level AND, mostly used with masks to cut out regions.

Geometric transformations

Q13. How do you resize an image while keeping the aspect ratio? ★ Most asked

Calculate the new height from the scale of the width.

```
h, w = img.shape[:2]
new_w = 400
new_h = int(h * new_w / w)
small = cv2.resize(img, (new_w, new_h), interpolation=cv2.INTER_AREA)
```

Note `cv2.resize` takes `(width, height)`. Use `INTER_AREA` to shrink and `INTER_LINEAR`/`INTER_CUBIC` to enlarge.

Q14. How do you rotate an image? ★ Most asked

```
h, w = img.shape[:2]
M = cv2.getRotationMatrix2D((w // 2, h // 2), 45, 1.0)  # center, angle, scale
rotated = cv2.warpAffine(img, M, (w, h))
```

For 90° steps, `cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)` is simpler. To avoid cutting corners, compute a bigger output size from the rotation matrix's cos/sin values.

Q15. How do you flip and crop an image?

Flip: `cv2.flip(img, 1)` (horizontal), `0` (vertical), `-1` (both). Crop with slicing: `img[50:200, 100:300]`.

Q16. What is the difference between affine and perspective transformation?

An **affine** transform (`warpAffine`) keeps parallel lines parallel: translation, rotation, scaling, shearing; it needs 3 points. A **perspective** transform (`warpPerspective`) can change parallel lines, like straightening a photo of a document taken at an angle; it needs 4 points (`getPerspectiveTransform`).

Filtering and blurring

Q17. What is blurring and why is it used? ★ Most asked

Blurring smooths an image by averaging nearby pixels. It reduces noise and small details before thresholding or edge detection.

- `cv2.blur(img, (5,5))`: simple average.
- `cv2.GaussianBlur(img, (5,5), 0)`: weighted average; most common.
- `cv2.medianBlur(img, 5)`: best for salt-and-pepper noise.
- `cv2.bilateralFilter(img, 9, 75, 75)`: smooths while keeping edges sharp (slower).

Q18. What is a kernel (convolution)?

A kernel is a small matrix (like 3×3) that slides over the image. At each position, pixel values are multiplied by the kernel and summed to make the new pixel. Different kernels blur, sharpen or detect edges. Custom kernels use `cv2.filter2D`.

Q19. Why must the kernel size be odd?

An odd size (3, 5, 7) has a clear center pixel, so the output aligns with the input pixel being processed.

Thresholding

Q20. What is thresholding? ★ Most asked

Converting a grayscale image into a binary (black and white) image: pixels above a threshold become white, others black. Used to separate objects from the background.

```
_, th = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
```

Q21. What is the difference between simple, adaptive and Otsu thresholding? ★ Most asked

- **Simple (global)**: one fixed value for the whole image. Fails with uneven lighting.
- **Adaptive**: calculates a different threshold for each small region, so it handles shadows and uneven lighting (great for scanned documents).

```
th = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY, 11, 2)
```

- **Otsu**: automatically finds the best global threshold from the histogram: `cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)`.

Edges, morphology and contours

Q22. What is edge detection? Explain Canny. ★ Most asked

Edges are places where brightness changes sharply, often object boundaries. **Canny** is the most popular detector. Its steps:

1. Reduce noise with a Gaussian blur.

2. Find intensity gradients (with Sobel).

3. Non-maximum suppression to thin edges to 1 pixel.

4. Double threshold + hysteresis: keep strong edges and weak edges connected to strong ones.

```
edges = cv2.Canny(gray, 100, 200)
```

Q23. What are Sobel and Laplacian operators?

Both find edges using derivatives. **Sobel** finds gradients in the X or Y direction separately (good for direction). **Laplacian** uses the second derivative in all directions at once but is more sensitive to noise.

Q24. What are erosion and dilation? ★ Most asked

Morphological operations on binary images using a kernel.

- **Erosion** shrinks white regions; removes small white noise and separates touching objects.
- **Dilation** grows white regions; fills small holes and joins broken parts.

```
kernel = np.ones((5,5), np.uint8)
eroded  = cv2.erode(mask, kernel, iterations=1)
dilated = cv2.dilate(mask, kernel, iterations=1)
```

Q25. What are opening and closing?

- **Opening** = erosion then dilation; removes small white noise dots.
- **Closing** = dilation then erosion; fills small black holes inside objects.

`cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)`.

Q26. What are contours and how do you find them? ★ Most asked

Contours are curves joining continuous boundary points of the same colour or intensity, i.e. object outlines. Use them on a binary image (after thresholding or Canny).

```
contours, hierarchy = cv2.findContours(th, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
cv2.drawContours(img, contours, -1, (0, 255, 0), 2)
for c in contours:
    area = cv2.contourArea(c)
    x, y, w, h = cv2.boundingRect(c)
```

Uses: counting objects, shape detection, measuring size.

Q27. How do you detect shapes like triangles and rectangles?

Find contours, then simplify each with `cv2.approxPolyDP(c, 0.02 * cv2.arcLength(c, True), True)`. Count the corner points: 3 = triangle, 4 = rectangle/square (check the aspect ratio), many = circle.

Q28. What are image moments?

Weighted averages of pixel intensities used to describe a shape: area, centroid (center of mass) and orientation. `M = cv2.moments(c)`, center `cx = M["m10"]/M["m00"]`, `cy = M["m01"]/M["m00"]`.

Lines, circles and features

Q29. What is the Hough Transform? ★ Most asked

A technique to detect shapes like lines and circles, even if they're broken or partly hidden.

```
lines = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=50, minLineLength=50,
maxLineGap=10)
circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp=1.2, minDist=30, param1=100,
param2=30)
```

Used for lane detection in self-driving cars and finding coins or pupils.

Q30. What is template matching?

Finding where a small template image appears in a bigger image by sliding it and comparing similarity.

```
res = cv2.matchTemplate(img, template, cv2.TM_CCOEFF_NORMED)
_, max_val, _, max_loc = cv2.minMaxLoc(res)
```

It doesn't handle rotation or scale changes well.

Q31. What are feature detectors? Compare SIFT, SURF and ORB. ★ Most asked

Feature detectors find distinctive keypoints (corners, blobs) and descriptors that can be matched between images.

- **SIFT**: very accurate, scale and rotation invariant, slower. Patent expired, now free in OpenCV.
- **SURF**: faster than SIFT but still patented (in contrib only).
- **ORB**: fast, free, good for real-time use and mobile; uses binary descriptors.

```
orb = cv2.ORB_create()
kp, des = orb.detectAndCompute(gray, None)
```

Q32. How do you match features between two images?

Detect keypoints and descriptors in both images, then match them with a **BFMatcher** (brute force) or **FLANN** (faster for large sets). Use Lowe's ratio test to keep only good matches. Used in panorama stitching and object recognition.

Q33. What is histogram equalization?

A method to improve contrast by spreading out the most frequent intensity values. `cv2.equalizeHist(gray)`. **CLAHE** (`cv2.createCLAHE`) does it in small tiles, avoiding over-brightening; good for low-light and medical images.

Q34. What is an image histogram?

A graph showing how many pixels have each intensity value (0-255). It tells if an image is too dark, too bright or low-contrast. `cv2.calcHist([gray], [0], None, [256], [0, 256])`.

Video and detection

Q35. How do you capture video from a webcam? ★ Most asked

```
cap = cv2.VideoCapture(0)       # 0 = default camera, or a file path
while True:
    ret, frame = cap.read()
    if not ret:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    cv2.imshow("Live", gray)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
cap.release()
```

```
cv2.destroyAllWindows()
```

Save video with `cv2.VideoWriter`.

Q36. How do you detect faces with OpenCV? ★ Most asked

The classic method uses **Haar cascades**:

```
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
"haarcascade_frontalface_default.xml")
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5)
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x + w, y + h), (255, 0, 0), 2)
```

Haar cascades are fast but miss tilted faces. For better accuracy use the DNN module with a pre-trained model, or YuNet (`cv2.FaceDetectorYN`).

Q37. What is the difference between face detection and face recognition?

Detection finds **where** faces are in an image. Recognition identifies **who** the face belongs to, by comparing face features (embeddings) with a known database (e.g. LBPH in OpenCV, or deep models like FaceNet).

Q38. How do you do simple motion detection?

Take the difference between consecutive frames (or a background frame) with `cv2.absdiff`, convert to gray, blur, threshold, dilate, then find contours. Large contours mean movement. `cv2.createBackgroundSubtractorMOG2()` is a more robust method.

Q39. What is object tracking? Name some trackers.

Tracking follows an object across video frames after it's detected once, which is faster than detecting in every frame. OpenCV has trackers like **KCF**, **CSRT** (accurate) and **MOSSE** (very fast). Optical flow (Lucas-Kanade) tracks points between frames.

Q40. What is optical flow?

The pattern of apparent motion of pixels between two frames. **Lucas-Kanade** (`cv2.calcOpticalFlowPyrLK`) tracks a few chosen points (sparse); **Farneback** computes motion for every pixel (dense). Used for motion analysis and video stabilisation.

Q41. What is the DNN module in OpenCV?

`cv2.dnn` loads and runs pre-trained deep learning models (ONNX, TensorFlow, Caffe, Darknet/YOLO) for detection, classification and segmentation. It does inference only, not training.

```
net = cv2.dnn.readNetFromONNX("model.onnx")
blob = cv2.dnn.blobFromImage(img, 1/255.0, (640, 640), swapRB=True)
net.setInput(blob)
out = net.forward()
```

Q42. How do you use OpenCV with YOLO?

Load a YOLO model in ONNX format with `cv2.dnn`, or (more commonly today) use the Ultralytics YOLO library for detection and OpenCV for reading frames, drawing boxes and showing results. Post-process with confidence filtering and **non-maximum suppression** (`cv2.dnn.NMSBoxes`) to remove duplicate boxes.

Q43. What is non-maximum suppression (NMS)?

When a detector finds many overlapping boxes for the same object, NMS keeps the box with the highest confidence and removes others that overlap it too much (high IoU).

Q44. What is image segmentation? Name some methods.

Splitting an image into meaningful regions (object vs background). Methods: thresholding, contours, **watershed** (for touching objects), **GrabCut** (foreground extraction with a rough box), k-means colour clustering, and deep models (U-Net, Mask R-CNN).

Q45. What is camera calibration?

Finding a camera's internal parameters (focal length, optical center) and lens distortion using pictures of a chessboard pattern (`cv2.findChessboardCorners`, `cv2.calibrateCamera`). It's needed to remove distortion and to measure real-world distances.

Q46. How would you build a document scanner with OpenCV?

Gray → blur → Canny → find the largest 4-corner contour (the paper) → order the corners → perspective transform to a flat top-down view → adaptive threshold for a clean scanned look.

Q47. How would you speed up a slow OpenCV application?

Resize frames to a smaller size, process only the ROI, convert to grayscale early, skip frames or use tracking instead of detecting every frame, use vectorised NumPy instead of Python loops, use multithreading for camera reading, and use GPU (CUDA) or lighter models.

Q48. How would you improve results on low-light images?

Use CLAHE or histogram equalization, gamma correction (brightening dark areas with a lookup table), denoising (`cv2.fastNlMeansDenoisingColored`), and work in the V channel of HSV.

Q49. What is image inpainting?

Filling missing or unwanted areas using nearby pixels. `cv2.inpaint(img, mask, 3, cv2.INPAINT_TELEA)` can remove scratches, text or small objects.

More frequently asked

Q50. What is the difference between `cv2.imshow` and `plt.imshow`?

`cv2.imshow` opens a native window and expects BGR images; it needs `waitKey`. `plt.imshow` (matplotlib) is used in notebooks and expects RGB, so convert first. In Jupyter or Colab, `cv2.imshow` usually doesn't work.

Q51. How do you draw shapes and text on an image?

```
cv2.line(img, (0, 0), (200, 200), (255, 0, 0), 2)
cv2.rectangle(img, (50, 50), (150, 150), (0, 255, 0), 3)
cv2.circle(img, (100, 100), 40, (0, 0, 255), -1)   # -1 = filled
cv2.putText(img, "Hello", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)
```

Colours are in BGR order.

Q52. What is an image pyramid?

A set of the same image at decreasing resolutions (`cv2.pyrDown`, `cv2.pyrUp`). Used to detect objects at different sizes, in image blending and in coarse-to-fine optical flow.